

On This Page >



01 Clan Modules

Status

Accepted

Context

Current state as of writing:

To define a service in Clan, you need to define two things:

- `clanModule` - defined by module authors
- `inventory` - defined by users

The `clanModule` is currently a plain NixOS module. It is conditionally imported into each machine depending on the `service` and `role`.

A `role` is a function of a machine within a service. For example in the `backup` service there are `client` and `server` roles.

The `inventory` contains the settings for the user/consumer of the module. It describes what `services` run on each machine and with which `roles`.

Additionally, any `service` can be instantiated multiple times.

This ADR proposes that we change how to write a `clanModule`. The `inventory` should get a new attribute called `instances` that allow for configuration of these modules.

Status Quo

In this example the user configures 2 instances of the `networking` service:

The user defines

```

1 {
2   inventory.services = {
3     # anything inside an instance is instance specific
4     networking."instance1" = {
5       roles.client.tags = [ "all" ];
6       machines.foo.config = { ... /* machine specific settings */ };
7
8       # this will not apply to `clients` outside of `instance1`
9       roles.client.config = { ... /* client specific settings */ };
10    };
11    networking."instance2" = {
12      roles.server.tags = [ "all" ];
13      config = { ... /* applies to every machine that runs this instance
14    };
15  };
16 }
```

The module author defines:

```

1 # networking/roles/client.nix
2 { config, ... }:
3 let
4   instances = config.clan.inventory.services.networking or { };
5
6   serviceConfig = config.clan.networking;
7 in {
8   ## Set some nixos options
9 }
```

Problems

Problems with the current way of writing clanModules:

1. No way to retrieve the config of a single service instance, together with its name.
2. Directly exporting a single, anonymous nixosModule without any intermediary attribute layers doesn't leave room for exporting other inventory resources such as potentially `vars` or `homeManagerConfig`.

3. Can't access multiple config instances individually. Example:

```
1 inventory = {
2   services = {
3     network.c-base = {
4       instanceConfig.ips = {
5         mors = "172.139.0.2";
6       };
7     };
8     network.gg23 = {
9       instanceConfig.ips = {
10        mors = "10.23.0.2";
11      };
12    };
13  };
14 };
```

This doesn't work because all instance configs are applied to the same namespace. So this results in a conflict currently. Resolving this problem means that new inventory modules cannot be plain nixos modules anymore. If they are configured via `instances` / `instanceConfig` they cannot be configured without using the inventory. (There might be ways to inject `instanceConfig` but that requires knowledge of inventory internals)

4. Writing modules for multiple instances is cumbersome. Currently the clanModule author has to write one or multiple `fold` operations for potentially every nixos option to define how multiple service instances merge into every single one option. The new idea behind this adr is to pull the common fold function into the outer context provide it as a common helper. (See the example below. `perInstance` analog to the well known `perSystem` of flake-parts)
5. Each role has a different interface. We need to render that interface into json-schema which includes creating an unnecessary test machine currently. Defining the interface at a higher level (outside of any machine context) allows faster evaluation and an isolation by design from any machine. This allows rendering the UI (options tree) of a service by just knowing the service and the corresponding roles without creating a dummy machine.
6. The interface of defining config is wrong. It is possible to define config that applies to multiple machine at once. It is possible to define config that applies to a machine as a hole. But this is wrong behavior because the options exist at the role level. So config must also always exist at the role level. Currently we merge options and config together but that may produce conflicts. Those module system conflicts are very hard to foresee since they depend on what roles exist at runtime.

Proposed Change

We will create a new module class which is defined by `_class = "clan.service"` ([documented here](#)).

Existing clan modules will still work by continuing to be plain NixOS modules. All new modules can set `_class = "clan.service";` to use the proposed features.

In short the change introduces a new module class that makes the currently necessary folding of `clan.service`s `instances` and `roles` a common operation. The module author can define the inner function of the fold operations which is called a `clan.service` module.

There are the following attributes of such a module:

`roles.<roleName>.interface`

Each role can have a different interface for how to be configured. I.e.: A `client` role might have different options than a `server` role.

This attribute should be used to define `options`. (Not `config` !)

The end-user defines the corresponding `config`.

This submodule will be evaluated for each `instance role` combination and passed as argument into `perInstance`.

This submodules `options` will be evaluated to build the UI for that module dynamically.

Result attributes

Some common result attributes are produced by modules of this proposal, those will be referenced later in this document but are commonly defined as:

- `nixosModule` A single nixos module. (`{config, ...}:{ environment.systemPackages = []; }`)
- `services.<serviceName>` An attribute set of `_class = clan.service`. Which contain the same thing as this whole ADR proposes.
- `vars` To be defined. Reserved for now.

`roles.<roleName>.perInstance`

This acts like a function that maps over all `service instances` of a given `role`. It produces the previously defined **result attributes**.

I.e. This allows to produce multiple `nixosModules` one for every instance of the service. Hence making multiple `service instances` convenient by leveraging the module-system merge behavior.

`perMachine`

This acts like a function that maps over all `machines` of a given `service`. It produces the previously defined **result attributes**.

I.e. this allows to produce exactly one `nixosModule` per `service`. Making it easy to set nixos-options only once if they have a one-to-one relation to a service being enabled.

Note: `lib.mkIf` can be used on i.e. `roleName` to make the scope more specific.

`services.<serviceName>`

This allows to define nested services. i.e the service `backup` might define a nested service `ssh` which sets up an ssh connection.

This can be defined in `perMachine` and `perInstance`

- For Every `instance` a given `service` may add multiple nested `services`.
- A given `service` may add a static set of nested `services`; Even if there are multiple instances of the same given service.

Q: Why is this not a top-level attribute? A: Because nested service definitions may also depend on a `role` which must be resolved depending on `machine` and `instance`. The top-level module doesn't know anything about machines. Keeping the service layer machine agnostic allows us to build the UI for a module without adding any machines. (One of the problems with the current system)

`zerotier/default.nix`

```
1 # Some example module
2 {
3   _class = "clan.service";
```

```

4
5 # Analog to flake-parts 'perSystem' only that it takes instance
6 # The exact arguments will be specified and documented along with the
7 roles.client.perInstance = {
8     # attrs : settings of that instance
9     settings,
10    # string : name of the instance
11    instanceName,
12    # { name :: string , roles :: listOf string; }
13    machine,
14    # { {roleName} :: { machines :: listOf string; } }
15    roles,
16    ...
17 };
18 {
19     # Return a nixos module for every instance.
20     # The module author must be aware that this may return multiple modules
21     nixosModule = {
22         config.debug."${instanceName}-client" = instanceConfig;
23     };
24 };
25 # Function that is called once for every machine with the role "client"
26 # Receives at least the following parameters:
27 #
28 # machine :: { name :: String, roles :: listOf string; }
29 # Name of the machine
30 #
31 # instances :: { instanceName :: { roleName :: { machines :: [ string
32 # Resolved roles
33 # Same type as currently in `clan.inventory.services.<ServiceName>.<Inst
34 #
35 # The exact arguments will be specified and documented along with the
36 perMachine = {machine, instances, ... }: {
37     nixosModule =
38         { lib, ... }:
39         {
40             # Some shared code should be put into a shared file
41             # Which is then imported into all/some roles
42             imports = [
43                 ../shared.nix
44             ] ++
45             (lib.optional (builtins.elem "client" machine.roles)
46                 {
47                     options.debug = lib.mkOption {
48                         type = lib.types.attrsOf lib.types.raw;
49                     };
50                 });
51         };
52 };

```

53 }

Inventory.instances

This document also proposes to add a new attribute to the inventory that allow for exclusive configuration of the new modules. This allows to better separate the new and the old way of writing and configuring modules. Keeping the new implementation more focussed and keeping existing technical debt out from the beginning.

The following thoughts went into this:

- Getting rid of `<serviceName>`: Using only the attribute name (plain string) is not sufficient for defining the source of the service module. Encoding meta information into it would also require some extensible format specification and parser.
- removing instanceConfig and machineConfig: There is no such config. Service configuration must always be role specific, because the options are defined on the role.
- renaming `config` to `settings` or similar. Since `config` is a module system internal name.
- Tags and machines should be an attribute set to allow setting `settings` on that level instead.

```

1 {
2   inventory.instances = {
3     "instance1" = {
4       # Allows to define where the module should be imported from.
5       module = {
6         input = "clan-core";
7         name = "borgbackup";
8       };
9       # settings that apply to all client machines
10      roles.client.settings = {};
11      # settings that apply to the client service of machine with name <machineName>
12      # There might be a server service that takes different settings on
13      roles.client.machines.<machineName>.settings = {};
14      # settings that apply to all client-instances with tag <tagName>
15      roles.client.tags.<tagName>.settings = {};
16    };
17    "instance2" = {
18      # ...
19    };
20  };
21 }
22

```

Iteration note

We want to implement the system as described. Once we have sufficient data on real world use-cases and modules we might revisit this document along with the updated implementation.

Real world example

The following module demonstrates the idea in the example of *borgbackup*.

```

1 {
2   _class = "clan.service";
3
4   # Define the 'options' of 'settings' see argument of perInstance
5   roles.server.interface =
6     { lib, ... }:
7     {
8       options = lib.mkOption {
9         type = lib.types.str;
10        default = "/var/lib/borgbackup";
11        description = ''
12          The directory where the borgbackup repositories are stored.
13        '';
14      };
15    };
16
17   roles.server.perInstance =
18     {
19       instanceName,
20       settings,
21       roles,
22       ...
23     }:
24     {
25       nixosModule =
26         { config, lib, ... }:
27         let
28           dir = config.clan.core.settings.directory;
29           machineDir = dir + "/vars/per-machine/";
30           allClients = roles.client.machines;
31         in
32         {
33           # services.borgbackup is a native nixos option
34           config.services.borgbackup.repos =
35             let

```



```

36     borgbackupIpMachinePath = machine: machineDir + machine + '
37
38     machinesMaybeKey = builtins.map (
39         machine:
40         let
41             fullPath = borgbackupIpMachinePath machine;
42         in
43         if builtins.pathExists fullPath then
44             machine
45         else
46             lib.warn ''
47                 Machine ${machine} does not have a borgbackup key at
48                 run `clan vars generate ${machine}` to generate it.
49             '' null
50     ) allClients;
51
52     machinesWithKey = lib.filter (x: x != null) machinesMaybeKey
53
54     hosts = builtins.map (machine: {
55         name = instanceName + machine;
56         value = {
57             path = "${settings.directory}/${machine}";
58             authorizedKeys = [ (builtins.readFile (borgbackupIpMachinePath
59             )
60         }) machinesWithKey;
61     in
62     if (builtins.listToAttrs hosts) != [ ] then builtins.listToAttrs
63     };
64 };
65
66 roles.client.interface =
67 { lib, ... }:
68 {
69     # There might be a better interface now. This is just how clan borg
70     options.destinations = lib.mkOption {
71         type = lib.types.attrsOf (
72             lib.types.submodule (
73                 { name, ... }:
74                 {
75                     options = {
76                         name = lib.mkOption {
77                             type = lib.types.strMatching "[a-zA-Z0-9._-]+$";
78                             default = name;
79                             description = "the name of the backup job";
80                         };
81                         repo = lib.mkOption {
82                             type = lib.types.str;
83                             description = "the borgbackup repository to backup to";
84                         };

```

```

85         rsh = lib.mkOption {
86             type = lib.types.nullOr lib.types.str;
87             default = null;
88             defaultText = "ssh -i \${config.clan.core.vars.generatorKey}";
89             description = "the rsh to use for the backup";
90         };
91     };
92 }
93 )
94 );
95 default = { };
96 description = ''
97     destinations where the machine should be backed up to
98 '';
99 };
100
101 options.exclude = lib.mkOption {
102     type = lib.types.listOf lib.types.str;
103     example = [ "*.pyc" ];
104     default = [ ];
105     description = ''
106         Directories/Files to exclude from the backup.
107         Use * as a wildcard.
108     '';
109 };
110 };
111 roles.client.perInstance =
112 {
113     instanceName,
114     roles,
115     machine,
116     settings,
117     ...
118 }:
119 {
120     nixosModule =
121     {
122         config,
123         lib,
124         pkgs,
125         ...
126     }:
127     let
128         allServers = roles.server.machines;
129
130         # machineName = config.clan.core.settings.machine.name;
131
132         # cfg = config.clan.borgbackup;
133         preBackupScript = ''

```

```

134 declare -A preCommandErrors
135
136 ${lib.concatMapStringsSep "\n" (
137     state:
138     lib.optionalString (state.preBackupCommand != null) ''
139     echo "Running pre-backup command for ${state.name}"
140     if ! /run/current-system/sw/bin/${state.preBackupCommand}
141     preCommandErrors["${state.name}"]=1
142     fi
143     ''
144 ) (lib.attrValues config.clan.core.state)}
145
146 if [[ '${preCommandErrors[@]} -gt 0 ]]; then
147     echo "pre-backup commands failed for the following services:
148     for state in "${!preCommandErrors[@]}"; do
149         echo "  $state"
150     done
151     exit 1
152 fi
153 '';
154
155 destinations =
156 let
157     destList = builtins.map (serverName: {
158         name = "${instanceName}-${serverName}";
159         value = {
160             repo = "borg@${serverName}:/var/lib/borgbackup/${machin
161             rsh = "ssh -i ${
162                 config.clan.core.vars.generators."borgbackup-${instan
163             } -o StrictHostKeyChecking=no -o UserKnownHostsFile=/de
164             } // settings.destinations.${serverName};
165         }) allServers;
166 in
167     (builtins.listToAttrs destList);
168 in
169 {
170     config = {
171         # Derived from the destinations
172         systemd.services = lib.mapAttrs' (
173             _ : dest:
174             lib.nameValuePair "borgbackup-job-${instanceName}-${dest.na
175             # since borgbackup mounts the system read-only, we need
176             serviceConfig.ExecStartPre = [
177                 ''+${pkgs.writeShellScript "borgbackup-job-${dest.name}
178             ];
179         }
180     ) destinations;
181
182     services.borgbackup.jobs = lib.mapAttrs (_destinationName: de

```

```

183     paths = lib.unique (
184         lib.flatten (map (state: state.folders) (lib.attrValues (
185             ));
186     exclude = settings.exclude;
187     repo = dest.repo;
188     environment.BORG_RSH = dest.rsh;
189     compression = "auto,zstd";
190     startAt = "*-*-* 01:00:00";
191     persistentTimer = true;
192
193     encryption = {
194         mode = "repokey";
195         passCommand = "cat ${config.clan.core.vars.generators."borg";
196     };
197
198     prune.keep = {
199         within = "1d"; # Keep all archives from the last day
200         daily = 7;
201         weekly = 4;
202         monthly = 0;
203     };
204 }) destinations;
205
206 environment.systemPackages = [
207     (pkgs.writeShellApplication {
208         name = "borgbackup-create";
209         runtimeInputs = [ config.systemd.package ];
210         text = ''
211             ${lib.concatMapStringsSep "\n" (dest: ''
212                 systemctl start borgbackup-job-${dest.name}
213                 '' ) (lib.attrValues destinations)}
214             '';
215     })
216     (pkgs.writeShellApplication {
217         name = "borgbackup-list";
218         runtimeInputs = [ pkgs.jq ];
219         text = ''
220             (${
221                 lib.concatMapStringsSep "\n" (
222                     dest:
223                     # we need yes here to skip the changed url verification
224                     'echo y | /run/current-system/sw/bin/borg-job-${dest.name}
225                     ) (lib.attrValues destinations)
226                 }) | jq -s 'add // []'
227             '';
228     })
229     (pkgs.writeShellApplication {
230         name = "borgbackup-restore";
231         runtimeInputs = [ pkgs.gawk ];

```

```

232         text = ''
233         cd /
234         IFS=: read -ra FOLDER <<< "'${FOLDERS-}"
235         job_name=$(echo "$NAME" | awk -F::: '{print $1}')
236         backup_name='${NAME#"${job_name}":}'
237         if [[ ! -x /run/current-system/sw/bin/borg-job-"$job_name" ]]; then
238             echo "borg-job-$job_name not found: Backup name is invalid"
239             exit 1
240         fi
241         echo y | /run/current-system/sw/bin/borg-job-"$job_name"
242     '';
243 })
244 ];
245 # every borgbackup instance adds its own vars
246 clan.core.vars.generators."borgbackup-${instanceName}" = {
247     files."borgbackup.ssh.pub".secret = false;
248     files."borgbackup.ssh" = { };
249     files."borgbackup.repokey" = { };
250
251     runtimeInputs = [
252         pkgs.coreutils
253         pkgs.openssh
254         pkgs.xkcdpass
255     ];
256     script = ''
257         ssh-keygen -t ed25519 -N "" -f $out/borgbackup.ssh
258         xkcdpass -n 4 -d - > $out/borgbackup.repokey
259     '';
260 };
261 };
262 };
263 };
264
265 perMachine = {
266     nixosModule =
267     { ... }:
268     {
269         clan.core.backups.providers.borgbackup = {
270             list = "borgbackup-list";
271             create = "borgbackup-create";
272             restore = "borgbackup-restore";
273         };
274     };
275 };
276 }

```

Prior-art

- <https://github.com/NixOS/nixops>
- <https://github.com/infinisil/nixus>

← Previous

Architecture Decisions

Next →

02 Clan As Library

Need help? Reach out to [our community](#).